COM-480 DATA VISUALIZATION · EPFL · SPRING 2026

Second-Degree Connection Explorer

Process Book

Team

Ahmed Chaouachi 346447

Shin Urech 327245

Joyti Goel 325374

This book documents how our team designed, built, and iterated on a D3.js network visualization that answers one question: who is one meaningful mutual-friend step away from you, and why does the introduction matter? We trace the journey from first sketches to a fully interactive static site, explaining every major design decision and the data challenges we encountered along the way.

CONTENTS

THE QUESTION AND THE DATASET

Why second-degree connections?

Most social platforms show you who you already know. The interesting question is who you *could* know: the friends of your friends who share something meaningful with you but whom you have never met. These second-degree connections are the ones most likely to become real friendships, precisely because a trusted mutual friend can make the introduction.

Our central question is: **"Who is one meaningful mutual-friend step away from me, and why does it matter?"** This question guided every design decision, from the ring layout of the force graph to the three-screen Find a Match wizard.

The Last.fm UK User Graph

We used the Last.fm UK User Graph dataset (Zenodo DOI 10.5281/zenodo.10694369), a real anonymised social network scraped from the Last.fm music platform. The dataset contains undirected friendship edges and per-user demographic attributes.

75,969 connected users	**389,639** unique friendships	**5** median degree
166 median 2nd-degree reach	**43** median both-trait 2nd-degree	**99.9%** users based in UK

Each user record carries four demographic fields: country, age group (10s, 20s, 30s, 40s, 50s, 60+), gender, and playcount. Notably, the dataset does not contain a per-user listening history table, which ruled out music-taste matching as a filter (see page 6).

What the data taught us early

Our Milestone 1 EDA revealed two structural properties that shaped the entire design:

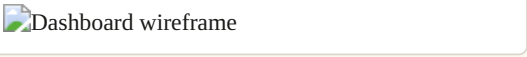
- **The second ring is large.** The median user has only 5 direct friends but 166 friends-of-friends, making the second degree the dominant opportunity space.
- **Attribute overlap is meaningful.** The median user shares both country and age group with 43 of their second-degree candidates. Across all direct friendships, 69% of friends share the ego's age group, confirming strong homophily even at the first hop.

> **Design consequence.** The visualization places the ego at the center, direct friends in the first ring, and candidates in the second ring. The sizing ratio (5 vs. 166) is why the second ring is always visually dominant and why candidate filtering matters.

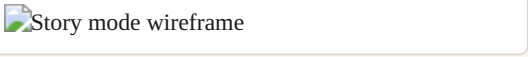
FROM SKETCHES TO PROTOTYPE

Milestone 1 wireframes

In Milestone 1 we produced two wireframes that defined the core interaction vocabulary. Both survive almost unchanged in the final product.



Dashboard wireframe: ego at center, two concentric rings, sidebar filters, ranked candidate list.



Story mode wireframe: A→B→C three-panel guided walkthrough of a single introduction path.

What changed from sketch to final

Sidebar candidate list	Moved below the graph into a separate ranked panel. The sidebar was too narrow for chip badges and score bars at typical viewport widths.
Music taste filter	Shown as a disabled "coming soon" checkbox. The dataset does not include per-user listening history, so the filter cannot be computed from available files alone.
Three-panel story mode	Became a single overlay with sequential slides rather than three side-by-side panels. The overlay avoids layout reflow and keeps the background graph visible.
Atlas section	Added below the Explorer: six animated D3 bar charts (degree distribution, second-degree distribution, shared-trait histogram, age groups, top countries, case comparison). Not in the original sketch but essential for contextualising the exemplar cases against the full dataset.
Find a Match wizard	Entirely new feature added in Milestone 3. A nav-bar button opens a three-screen modal: persona selection, search with a live force graph, and a path-reveal screen showing all N bridge paths simultaneously.

THE EXPLORER: DESIGN DECISIONS

Color encoding

Three node types carry the semantic weight of the visualization. The color assignment is intentional: teal (associated with the self/center) for the ego, sky-blue (calm, supportive) for bridge friends, coral (warm, inviting) for candidates.

Ego user (you) Bridge friends (1st hop) Candidates (2nd hop) Active path highlight

Ring layout rationale

The D3 force simulation uses `forceRadial` to assign each node group a target orbit radius: ego fixed at center, bridges at R=170px, candidates at R=325px. This keeps the two-hop structure legible even as physics forces allow natural spacing within each ring.

An important consequence: edges in this layout carry directional meaning even though the graph is undirected. Every edge visible on screen is a potential introduction path. This makes the graph readable as a story rather than as a hairball.

Candidate scoring and ranking

Each second-degree candidate is scored as:

```
score = (mutual friends × 2) + same_country + same_age_group + same_gender
```

Mutual friends are weighted double because they represent multiple independent introduction paths. Shared attributes add context but are secondary to structural connectivity. The slider cap (default 12 visible candidates) prevents the outer ring from becoming unreadable at high filter settings.

Bridge friend highlighting

Hovering a candidate dims all other nodes and draws glowing edges from the ego through the relevant bridge friends to that candidate. Clicking locks the selection and opens the detail panel, which names each bridge friend and explains which shared attributes make the match strong.

This interaction was the hardest to tune. Early versions highlighted too many nodes at once (all candidates reachable through the same bridge). The final design highlights only the path relevant to the hovered or selected candidate, keeping attention focused.

Filter logic: ANY vs. ALL

The Explorer offers a toggle between "match any checked trait" and "match all checked traits." This matters because the dataset is demographically homogeneous (99.9% UK, heavily 20s): "match all" with two attributes often returns the same result as "match any." Surfacing this to the user makes the homophily finding visible rather than hiding it behind a single mode.

STORY MODE AND FIND A MATCH

Story Mode

The M1 story wireframe showed A, B, and C in separate side-by-side panels. In implementation we collapsed this into a single full-screen overlay with sequential slides. The redesign has three advantages:

- The background graph remains visible, keeping spatial context.
- The overlay pattern (blur backdrop, slide-up animation) is already established in the UI, so Story Mode feels native rather than jarring.
- Sequential slides allow narrated pacing, which is closer to a presentation than a dashboard view.

Each story slide uses the same SVG helper functions as the rest of the application, so nodes and edges are rendered consistently across Explorer, Story Mode, and Find a Match.

Find a Match: a new feature for Milestone 3

Find a Match is a three-screen wizard accessible from the nav bar. It reframes the visualization from a researcher's tool into an emulation of a real friend-recommendation product.

Screen 1: Persona selection

The user picks which of the three exemplar ego users to "be." Each card shows age, gender, country, direct-friend count, and second-degree reach. This screen sets up the social context before any search begins.

Screen 2: Search with a live force graph

Instead of a static card grid, search results appear as a mini force graph inside the panel: ego at center, bridge friends in the middle ring, matching candidates in the outer ring. No edges are drawn by default. Hovering a candidate node draws the exact paths (ego to bridge, bridge to candidate) and dims all irrelevant nodes. This keeps the graph from being a visual mess while making the structural insight immediately tangible.

Filters (age group, gender, mutual friends count) narrow the candidate pool live; the graph re-renders with each change.

Screen 3: Path reveal

Clicking a candidate animates Screen 3 into view with a rise effect. The SVG shows a fan diagram: ego on the left, all N bridge friends stacked vertically in the middle, the candidate on the right. If there are 4 mutual friends, 4 paths are drawn simultaneously, making it immediately clear that the user has 4 different people who could make this introduction.

> **Design insight.** The fan diagram emerged from a key data finding: some candidates share 4 or more mutual friends. Showing only the "best" bridge (as the original design did) hides this richness. Multiple paths mean multiple low-risk ways to reach out, which is exactly the practical value of second-degree recommendations.

DATA CHALLENGES AND FINDINGS

Challenge 1: Music taste is not in the dataset

Our M1 sketches and the M2 writeup both envisioned a music-taste filter based on artist-tag overlap. The raw dataset does include an `ArtistTags` file (246 MB) and an `ArtistsMap` file, but there is no user-to-artist listening table in the available workspace. Without a table linking users to artists they have listened to, tag overlap cannot be computed at the user level.

We exposed this limitation honestly: the music-taste filter appears in the Explorer as a clearly labeled disabled checkbox, and the site's caveat strip explains the reason. This was the right call: a fake filter that returns arbitrary results would undermine trust in the entire visualization.

Challenge 2: Social homophily

The dataset is 99.9% UK users, making the country filter cosmetically interesting but analytically redundant. More subtly, we discovered that even with a 50-candidate pool per ego user, almost all second-degree candidates share the ego's own age group. This is social homophily: people tend to befriend demographically similar people, and that similarity propagates two hops out.

Rather than hiding this, we built it into the narrative. The Find a Match wizard makes homophily tangible: select a 10s ego user and all candidates are also 10s; select a 20s user and the same happens. The mutual-friends filter is the one dimension that genuinely differentiates candidates within the same demographic cluster.

> **Finding.** Across the three ego cases, 92% of finder candidates share the ego's age group (138 out of 150). The friend age-group match rate across the full dataset is 69%, meaning homophily is even stronger in the second-degree layer than in direct friendships. This is an honest data story, not a visualization flaw.

Challenge 3: Bridge candidates computed from a small pool

The Python build pipeline pre-computes the top 16 candidates per ego user for the Explorer graph. The Find a Match wizard needed 50 candidates for richer filtering. When we expanded the pool, we discovered that the `bridge_candidates` field on each direct friend only listed candidates from the original 16. Candidates in positions 17 to 50 had no bridge nodes in the graph and therefore showed no hover paths.

The fix was to build the bridge node set from the candidates' own `mutual_friend_ids` field rather than from the bridges' `bridge_candidates` list. This ensures every candidate in the larger pool has a bridge path available, regardless of which pool it was computed from.

Challenge 4: Pre-computation and exemplar selection

The full adjacency graph (75,969 nodes, 389,639 edges) cannot be loaded in a browser. We resolved this by selecting three exemplar ego users that each represent a distinct demographic profile and degree range, then pre-computing their full ego cases (16 Explorer candidates + 50 finder candidates each) in a Python build script. The results are baked into a single static JSON file and served from GitHub Pages with no backend. Focusing the interactive experience on three well-chosen personas is a deliberate storytelling decision: it keeps every interaction purposeful and lets the narrative stay focused on the second-degree concept rather than on data exploration for its own sake.

TECHNICAL IMPLEMENTATION

Architecture overview

The site is a fully static HTML/CSS/JavaScript application with no backend or build tool. D3.js v7 is loaded via CDN. Data is embedded as a JavaScript global (`window.MILESTONE_DATA`) in `docs/data.js` and is generated by a Python build pipeline. Deployment is via GitHub Pages from the `/docs` folder.

Python build pipeline

Raw data lives in `data/` (network edge list, user demographics). Two source modules in `src/` handle all graph computation:

- `music_data_utils.py`: loads and cleans the raw files using pandas, computes age groups, normalises country codes.
- `second_degree_utils.py`: builds the adjacency dict, computes second-degree reach per user, selects exemplar users, and assembles the full ego-case payload (user profile, direct friends, candidates, edges).

`milestone2/build_milestone2_assets.py` calls both modules, assembles the full JSON bundle, and writes it to `milestone2/data/analysis_summary.json`. A separate step copies this into `docs/data.js` as a JS global assignment. The live explorer currently ships three curated ego cases with ranked candidate slices while still reporting each ego's full second-degree reach from the processed data.

D3.js force graph

The network graph uses `d3.forceSimulation` with four forces: `forceRadial` (ring structure), `forceManyBody` (repulsion), `forceLink` (edge tension), and `forceCollide` (overlap prevention). The ego node is fixed at the SVG center via `fx`/`fy`.

SVG layers are ordered: ring guides, pulse ring, edges, nodes, labels, legend. This order ensures hover highlights are always visible and glowing edges render above static edges without z-index hacks.

Find a Match mini graph

The finder's mini graph uses the same D3 force simulation approach but runs 300 synchronous ticks (instead of an animated loop) to reach a stable layout immediately. Hover interactions are handled via D3 event listeners on SVG circle elements: `mouseenter` draws path lines and dims non-relevant nodes, `mouseleave` clears them. Clicking a candidate node triggers a CSS `riseUp` keyframe animation on Screen 3 before it is shown.

Static site advantages and trade-offs

Advantages

- Zero hosting cost (GitHub Pages)
- No server-side failures or latency
- Works offline after first load
- Trivial to share a direct URL

Considerations

- Three exemplar personas chosen by design, not arbitrary
- Candidate pool is pre-ranked, not computed live
- Rebuild required when source data changes
- Full graph traversal requires a backend or WebAssembly

PEER ASSESSMENT

Contributions by team member

The table below breaks down the primary responsibilities each team member owned across the project milestones. Contributions were collaborative throughout; this breakdown reflects who led each area.

Team member	Primary contributions
Ahmed Chaouachi 346447	Led the Python data pipeline: raw data ingestion, adjacency graph construction, second-degree computation, exemplar user selection, and the JSON build scripts. Implemented the D3 Atlas section (six animated bar charts). Performed the Milestone 1 EDA and drafted the quantitative findings (degree distributions, attribute match rates) that informed the scoring formula.
Shin Urech 327245	Led the D3 force graph implementation: forceSimulation tuning, radial ring layout, node and edge rendering, SVG layer ordering, drag behavior, and highlight/glow interactions. Implemented the bridge-friend detail panel and the animated candidate ranking. Responsible for the overall site architecture (static HTML/CSS/JS, GitHub Pages deployment).
Joyti Goel 325374	Led UI/UX design: the dark navy design system, CSS variables, typography, color palette, and responsive layout. Designed and implemented the Story Mode overlay. Designed and built the Find a Match wizard (persona selector, live mini force graph with hover-path reveal, multi-path fan diagram on Screen 3, all four filters). Wrote the process book.

Self-assessment

The visualization achieves all six MVP goals from the Milestone 2 plan: user selector, D3 force graph, attribute filters, candidate ranking, detail panel, and atlas charts. Milestone 3 adds Story Mode and the Find a Match wizard, both of which exceed the original scope.

The choice to build around three pre-computed exemplar personas was deliberate: each user was selected to represent a different age group and degree range, keeping the storytelling focused. A live traversal backend could expand this to any user in the dataset, but would introduce server infrastructure, latency, and hosting costs that conflict with the static-site requirement.

The music-taste filter remains unimplemented due to missing data, as documented honestly in the site's caveat strip and disabled filter UI. This is a data limitation, not a design or implementation failure.

> **GitHub repository:** github.com/ShinUrech/TheSocialGraph
> **Live site:** shinurech.github.io/TheSocialGraph/